

Python Interview Questions

LEVEL 1 - PYTHON FOUNDATIONS

1. Python Basics & Execution Model

Core Understanding

1. What is Python and why is it popular?
2. What are the major features of Python?
3. Why is Python called a high-level language?
4. What is dynamically typed behavior in Python?
5. What does interpreted language mean in Python?
6. How does Python execute code internally?
7. What is bytecode in Python?
8. What is the role of the Python Virtual Machine (PVM)?
9. Difference between compiler and interpreter.
10. Why is Python slower than C/C++?
11. What happens when a Python script runs?
12. What is the purpose of .pyc files?
13. What is the difference between CPython, PyPy, Jython, and IronPython?
14. What is the GIL (Global Interpreter Lock)?
15. How does Python handle multithreading?
16. What are identifiers and keywords?

17. What are comments and docstrings?
18. What is indentation and why is it important?
19. What are Python naming conventions?
20. What is PEP 8?

2. Variables, Objects & Memory

1. Why are Python variables references to objects?
2. Why is everything an object in Python?
3. Difference between mutable and immutable objects.
4. What is object identity?
5. What does id() function do?
6. What is reference counting?
7. How is memory managed in Python?
8. What is garbage collection?
9. What are cyclic references?
10. What is the gc module?
11. What is PyMalloc?
12. What causes memory leaks in Python?
13. What are shallow copy and deep copy?
14. Difference between copy() and deepcopy().
15. What are weak references?
16. Why is None a singleton?
17. What is string interning?
18. How are small integers optimized in Python?
19. What happens in multiple assignment?
20. How does Python handle memory differently from C/C++?
21. What is object lifecycle in Python?

22. What is stack memory vs heap memory?
23. How are Python objects stored internally?
24. What is memory fragmentation?
25. How can memory usage be optimized?

3. Data Types & Operators

Numbers

1. Difference between int, float, complex.
2. How does Python handle floating-point precision?
3. Why does $0.1 + 0.2 \neq 0.3$?
4. What is the Decimal module?
5. Difference between floor division and true division.

Strings

6. Why are strings immutable?
7. How are strings stored internally?
8. Difference between str, bytes, bytearray.
9. Common string methods.
10. What is Unicode?
11. What are raw strings?
12. What is string formatting?
13. Difference between %, format(), and f-strings.
14. What are escape characters?

Collections

15. Difference between list, tuple, set, dictionary.
16. Why are lists mutable and tuples immutable?
17. Why are sets unordered?
18. How are dictionaries implemented internally?
19. What is hashing?
20. Why must dictionary keys be immutable?
21. What is a frozenset?
22. What is tuple unpacking?
23. What is extended unpacking?
24. Why does slicing create new objects?
25. Why does range() not return a list?
26. What is negative indexing?
27. What is sequence packing/unpacking?

Operators

28. Difference between == and is.
29. Difference between logical and bitwise operators.
30. What is operator precedence?
31. What is short-circuit evaluation?
32. What are membership operators?
33. What are identity operators?
34. What are augmented assignment operators?
35. Why can += behave differently on mutable objects?

LEVEL 2 - CONTROL FLOW & FUNCTIONS

4. Conditional Statements & Loops

1. How does if-elif-else work?
2. What are truthy and falsy values?
3. How does chained comparison work?
4. What is ternary conditional expression?
5. Difference between for loop and while loop.
6. Why does Python not need C-style for loops?
7. What is enumerate()?
8. What is zip()?
9. How does break work?
10. How does continue work?
11. What is pass statement?
12. What is loop else clause?
13. How does for...else work?
14. Difference between while True and conditional loops.
15. What are nested loops?
16. How can loops be optimized?
17. What are comprehensions?
18. Difference between list comprehension and generator expression.
19. What are nested comprehensions?
20. How does match-case work in Python 3.10+?
21. Difference between match-case and if-elif.

22. How do `all()` and `any()` work?
23. What are iterator protocols?
24. What is iterable vs iterator?
25. How does `iter()` and `next()` work?

5. Functions

Function Basics

1. What is a function?
2. Difference between built-in and user-defined functions.
3. Difference between `return` and `print`.
4. Can Python return multiple values?
5. What are docstrings?
6. Difference between `def` and `lambda`.
7. What are pure and impure functions?
8. What is recursion?
9. Difference between recursion and iteration.
10. What is tail recursion?

Function Arguments

11. Positional arguments.
12. Keyword arguments.
13. Default arguments.
14. Why should mutable defaults be avoided?
15. `*args` and `**kwargs`.
16. Keyword-only arguments.
17. Positional-only arguments.

18. Argument unpacking.
19. Function annotations.
20. Type hints in functions.

Scope & Closures

21. What is variable scope?
22. What is LEGB rule?
23. Difference between local and global variables.
24. What does global keyword do?
25. What does nonlocal keyword do?
26. What are closures?
27. How do closures retain state?
28. What are higher-order functions?
29. Real-world uses of closures.
30. Relationship between closures and decorators.

Functional Programming

31. What is lambda?
32. How does map() work?
33. How does filter() work?
34. What is reduce()?
35. Difference between map/filter and comprehensions.
36. What are anonymous functions?
37. What are callback functions?
38. What is currying?
39. What is partial() from functools?
40. What is singledispatch?

LEVEL 3 - ADVANCED PYTHON

6. Iterators, Generators & Decorators

Iterators

1. What is an iterator?
2. Difference between iterable and iterator.
3. How does StopIteration work?
4. How to build custom iterators?
5. What is lazy evaluation?

Generators

6. What is a generator?
7. Difference between generators and normal functions.
8. How does yield work?
9. Why are generators memory efficient?
10. What is yield from?
11. What are generator expressions?
12. What is send() in generators?
13. How can generators create infinite sequences?
14. How does generator state persist?
15. Can generators be reused?
16. What is close() in generators?
17. What are async generators?

Decorators

18. What is a decorator?
19. How does `@decorator` syntax work?
20. What is `functools.wraps`?
21. How to create decorators with arguments?
22. Multiple decorators execution order.
23. Class decorators.
24. Function decorators.
25. Real-world uses of decorators.
26. Decorators in Flask/Django.
27. Memoization using decorators.
28. Caching with `lru_cache`.

7. Modules, Packages & Imports

1. What is a module?
2. What is a package?
3. Difference between module and package.
4. Absolute vs relative imports.
5. What is **name**?
6. Purpose of if **name** == **'main'**.
7. How does Python search for modules?
8. What is `sys.path`?
9. How can modules be reloaded?
10. What is `importlib`?
11. What are namespace packages?
12. What are built-in modules?

13. What is virtual environment?
14. What is pip?
15. What is requirements.txt?
16. What is pyproject.toml?
17. What are wheels and source distributions?
18. What is dependency management?
19. What is package versioning?
20. What are Python entry points?

LEVEL 4 - OBJECT ORIENTED

PROGRAMMING

8. OOP Fundamentals

1. What is OOP?
2. What are classes and objects?
3. Difference between class variables and instance variables.
4. What is self?
5. How object creation works internally?
6. Difference between **new** and **init**.
7. What is encapsulation?
8. What is inheritance?
9. What is polymorphism?
10. What is abstraction?
11. Method overriding.
12. Method overloading.

13. Multiple inheritance.
14. What is MRO?
15. How does `super()` work?
16. What are abstract classes?
17. What is the `abc` module?
18. What are interfaces in Python?
19. What is composition?
20. Composition vs inheritance.
21. What are static methods?
22. What are class methods?
23. Difference between `staticmethod` and `classmethod`.
24. What are magic methods?
25. What are dunder methods?

9. Advanced OOP Concepts

1. What is operator overloading?
2. What is method resolution order?
3. What are descriptors?
4. Difference between **`getattr`** and **`getattrattribute`**.
5. What is property decorator?
6. What is **`slots`**?
7. What are metaclasses?
8. What is metaprogramming?
9. What are data classes?
10. What are protocols in typing?
11. What are mixins?
12. What is duck typing?

13. What is monkey patching?
14. What are context managers?
15. Difference between **enter** and **exit**.
16. What is contextlib?
17. How does with statement work?
18. What are custom exceptions in OOP?
19. What is dependency injection?
20. What are SOLID principles in Python?

LEVEL 5 - ERROR HANDLING & DEBUGGING

10. Exception Handling

1. What is exception handling?
2. Difference between syntax errors and runtime errors.
3. What is try-except-finally?
4. What is else block in exceptions?
5. What is exception propagation?
6. Difference between Exception and BaseException.
7. What are built-in exceptions?
8. Difference between TypeError and ValueError.
9. Difference between KeyError and IndexError.
10. Difference between ImportError and ModuleNotFoundError.
11. What is raise?
12. What is assert?

13. Exception chaining using raise from.
14. How to create custom exceptions.
15. Why avoid bare except?
16. How to re-raise exceptions?
17. How to log exceptions?
18. Resource cleanup using finally.
19. Exception handling in recursion.
20. Exception handling in threads.
21. Real-world use cases of exceptions.
22. What is traceback?
23. What is pdb?
24. What is breakpoint()?
25. Difference between debugging and logging.

LEVEL 6 - FILES, DATABASES & APIs

11. File Handling

1. How does open() work?
2. File modes in Python.
3. Difference between text and binary files.
4. Reading CSV files.
5. Writing CSV files.
6. Reading JSON files.
7. Writing JSON files.
8. What is serialization?
9. What is pickle?

10. Difference between pickle and JSON.
11. Handling encoding issues.
12. Working with Excel files.
13. What is openpyxl?
14. How to work with zip files?
15. Directory handling using os module.
16. What is pathlib?
17. Difference between relative and absolute paths.
18. How to process large files efficiently?
19. What is buffering?
20. Streaming large files.

12. Database Connectivity

1. Connecting Python with MySQL.
2. Connecting Python with SQLite.
3. Difference between MySQL and SQLite.
4. What is a cursor object?
5. CRUD operations using Python.
6. What are transactions?
7. What is commit and rollback?
8. Parameterized queries.
9. SQL injection prevention.
10. Connection pooling.
11. ORM vs raw SQL.
12. What is SQLAlchemy?
13. Exporting DataFrames to SQL.
14. Database exception handling.

15. How to retrieve metadata from databases.
16. What are joins?
17. Connection testing strategies.
18. Logging database errors.
19. Batch inserts optimization.
20. Database indexing basics.

LEVEL 7 - NUMPY, PANDAS & DATA ANALYTICS

13. NumPy

1. What is NumPy?
2. Why is NumPy faster than lists?
3. What are ndarrays?
4. Array creation methods.
5. Difference between arange and linspace.
6. What is vectorization?
7. What is broadcasting?
8. Array reshaping.
9. Indexing and slicing.
10. Boolean masking.
11. copy() vs view().
12. flatten() vs ravel().
13. Aggregation functions.
14. Handling NaN values.

15. Random module in NumPy.
16. Matrix multiplication.
17. Difference between dot and matmul.
18. Universal functions (ufuncs).
19. Performance optimization in NumPy.
20. Memory layout in arrays.

14. Pandas

1. What is Pandas?
2. Difference between Series and DataFrame.
3. Reading CSV/Excel files.
4. DataFrame inspection methods.
5. loc vs iloc.
6. Filtering rows.
7. Handling missing values.
8. GroupBy operations.
9. Aggregation functions.
10. Merge, join, concat.
11. Sorting data.
12. Renaming columns.
13. Removing duplicates.
14. Working with datetime.
15. Pivot tables.
16. apply() vs map().
17. Lambda in Pandas.
18. Memory optimization in Pandas.
19. Chunk processing.

20. Time-series analysis basics.

15. Data Visualization

Matplotlib

1. What is Matplotlib?
2. Creating line plots.
3. Bar charts and histograms.
4. Pie charts.
5. Scatter plots.
6. Subplots.
7. Figure customization.
8. Saving plots.
9. Labels and legends.
10. Plot styling.

Seaborn

11. What is Seaborn?
12. Difference between Seaborn and Matplotlib.
13. Heatmaps.
14. Pairplots.
15. Histplots.
16. Boxplots.
17. Violin plots.
18. Categorical plots.
19. Distribution analysis.
20. Statistical visualization.

LEVEL 8 - DATA SCIENCE

FOUNDATIONS

16. Data Analysis & ML Foundations

1. What is EDA?
2. What is data wrangling?
3. Handling noisy data.
4. Handling missing values.
5. Feature engineering basics.
6. Encoding categorical variables.
7. Label encoding vs one-hot encoding.
8. Feature scaling.
9. Standardization vs normalization.
10. Handling imbalanced datasets.
11. What is scikit-learn?
12. Train-test split.
13. Overfitting vs underfitting.
14. Bias vs variance.
15. Cross-validation.
16. Evaluation metrics.
17. Real-time analytics.
18. Automating reports.
19. Memory optimization in datasets.
20. Deploying ML/data solutions.

LEVEL 9 - CONCURRENCY & ASYNC

PROGRAMMING

17. Multithreading, Multiprocessing & Asyncio

1. What is concurrency?
2. Difference between concurrency and parallelism.
3. CPU-bound vs IO-bound tasks.
4. What is threading?
5. What is multiprocessing?
6. What is asyncio?
7. What is coroutine?
8. async and await keywords.
9. What is event loop?
10. What is cooperative multitasking?
11. ThreadPoolExecutor vs ProcessPoolExecutor.
12. What are daemon threads?
13. What are race conditions?
14. What are locks?
15. Difference between Lock and RLock.
16. What is Semaphore?
17. What is Event object?
18. What is Queue for thread communication?
19. What is shared memory?
20. What is pickling in multiprocessing?
21. How to cancel asyncio tasks?

22. What is bounded concurrency?
23. Mixing blocking code with asyncio.
24. Performance benchmarking in concurrency.
25. How does the GIL impact concurrency?

LEVEL 10 - TESTING, DEBUGGING & CODE QUALITY

18. Testing & Debugging

1. What is unit testing?
2. Difference between unit, integration, and E2E testing.
3. What is unittest?
4. What is pytest?
5. unittest vs pytest.
6. What are fixtures?
7. What is mocking?
8. What is patching?
9. What are stubs and fakes?
10. What is code coverage?
11. What is CI/CD testing?
12. What is static analysis?
13. What is mypy?
14. What is flake8?
15. Logging best practices.
16. Capturing logs in tests.

17. Testing async code.
18. Profiling using cProfile.
19. What is tracemalloc?
20. Designing testable code.

LEVEL 11 - ADVANCED & EXPERT

PYTHON

19. Advanced Python Internals

1. What is metaprogramming?
2. How do metaclasses work?
3. What are descriptors?
4. Difference between **getattr** and **getattrattribute**.
5. What are slots?
6. How do data classes reduce boilerplate?
7. What is lexical scoping?
8. Risks of `eval()` and `exec()`.
9. Safe alternatives to `eval()`.
10. What is dynamic importing?
11. What is monkey patching?
12. What are protocols in typing?
13. What is structural typing?
14. What are generic types?
15. What are annotations?
16. What is reflection/introspection?

17. What is dependency injection?
18. What are plugins in Python?
19. What is dynamic dispatch?
20. What are Python internals interview questions asked in FAANG companies?

LEVEL 12 - SYSTEM DESIGN & PRODUCTION PYTHON

20. Production Engineering Concepts

1. How do you structure large Python projects?
2. What is clean architecture?
3. What are design patterns in Python?
4. Singleton pattern.
5. Factory pattern.
6. Observer pattern.
7. Strategy pattern.
8. Dependency injection pattern.
9. What is caching?
10. Redis integration basics.
11. What is API rate limiting?
12. Logging and monitoring strategies.
13. What is Docker?
14. Python inside containers.
15. Environment variable management.
16. Secrets management.

17. What are microservices?
18. REST API fundamentals.
19. What is FastAPI?
20. What is Flask?
21. What is Django?
22. Synchronous vs asynchronous APIs.
23. API authentication methods.
24. JWT authentication.
25. What is Celery?
26. Background task processing.
27. What is message queue?
28. Kafka basics.
29. How do you scale Python applications?
30. Production debugging strategies.